

Agentic AI — Complete Course Study Guide

LangChain v1 • LangGraph • RAG • Vectorless RAG • Deep Agents • Guardrails • LLM Evaluation • LLM Gateways

Built from the Krish Naik 10+ hour course structure, the linked GitHub notebooks/repositories, and production-oriented explanations.

Goal: understand the architecture, not just memorize framework APIs. Every module includes definitions, mental models, diagrams, practical examples, code patterns, interview notes, and production pitfalls.

Course map

Module	Video section	Core outcome
1	00:00:00	GenAI + Agentic AI foundations
2	00:02:31	LangChain v1 building blocks
3	02:35:12	LangGraph stateful orchestration
4	05:02:29	RAG end-to-end
5	07:10:43	Vectorless / PageIndex-style RAG
6	08:02:11	Deep Agents
7	08:45:43	Guardrails & safety
8	09:22:55	LLM / RAG evaluation
9	10:30:25	LLM Gateways

1. Foundations — GenAI, LLMs and Agentic AI

Definition

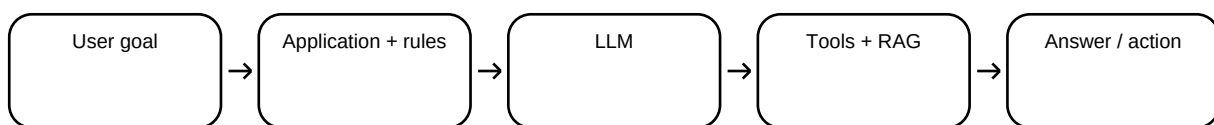
Generative AI produces new content such as text, code, summaries or structured data. An LLM is the model that predicts/generates tokens. An agentic system adds decision-making, tool use, state, observation and a loop so the system can work toward a goal instead of returning only one model response.

Mental model

LLM = brain; tools = hands; RAG = external knowledge; LangGraph = workflow/control plane; guardrails = safety layer; evals = quality measurement; gateway = model infrastructure.

Architecture

Production GenAI / Agentic AI



Example

A customer asks: “Check my order and cancel it if it is eligible.” A plain LLM can explain cancellation rules. An agent can authenticate, call an order API, inspect the result, check policy, ask for approval when required, call the cancellation API, and report the result.

Why not only an LLM?

An LLM alone does not automatically have private/current data, deterministic workflow control, permission boundaries, tool execution, durable state, observability, or reliable failure handling.

Study checkpoint: Explain this topic in 60 seconds, draw the architecture from memory, then name two failure modes and two production mitigations.

2. LangChain v1 — Application Building Blocks

Definition

LangChain is an application framework for connecting models, messages, prompts, tools, structured output and agent behavior. The course focuses on LangChain v1 concepts and middleware rather than older chain-centric APIs.

Core concepts

Models; messages; prompts; tools; agents; structured output; middleware; state/memory.

Architecture

LangChain building blocks



Code pattern

```
from langchain.agents import create_agent

def get_order_status(order_id: str) -> str:
    return f"Order {order_id}: shipped"

agent = create_agent(
    model=model,
    tools=[get_order_status],
    system_prompt="You are a support agent. Use tools when needed."
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Where is order 123?"}]
})
```

Interview distinction

LangChain is not an LLM. It is an orchestration/application layer. You can call an LLM directly; LangChain helps standardize how model calls, tools, prompts and agent execution are composed.

Study checkpoint: Explain this topic in 60 seconds, draw the architecture from memory, then name two failure modes and two production mitigations.

3. LangGraph — Stateful Agent Orchestration

Definition

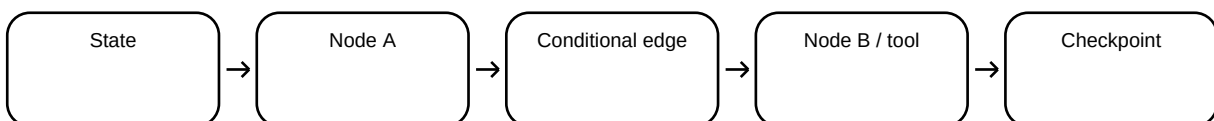
LangGraph models an agent/workflow as a graph. The three words to remember are State → Nodes → Edges. State carries shared information, nodes perform work, and edges decide what happens next. Checkpoints make execution resumable and durable.

Core concepts

State; nodes; edges; conditional edges; reducers; checkpoints; threads; persistence; human-in-the-loop; subgraphs; multi-agent patterns.

Architecture

LangGraph



Example

Refund agent: Start → read request → classify → eligibility check → if high value, human approval → refund API → response. API failure can route to retry/error handling.

Why LangGraph?

Use it when you need explicit branching, loops, retries, persistent state, HITL, fault recovery, or multi-agent coordination. Do not add a graph merely because it sounds advanced.

Study checkpoint: Explain this topic in 60 seconds, draw the architecture from memory, then name two failure modes and two production mitigations.

4. RAG — Retrieval-Augmented Generation

Definition

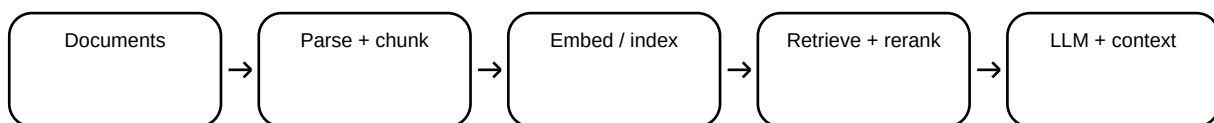
RAG retrieves relevant external information and supplies it to the LLM at query time. It is useful for private, changing, domain-specific or citation-sensitive knowledge without retraining the model.

Indexing vs querying

Indexing prepares the knowledge base. Querying uses a user question to retrieve evidence and generate an answer.

Architecture

RAG pipeline



Chunking

Fixed-size, recursive, semantic, structure-aware and table-aware chunking. There is no universal best size; evaluate against retrieval quality and the document/query type.

Retrieval quality

If the right evidence is not retrieved, fix parsing, chunking, embeddings, metadata filters, hybrid retrieval or reranking. If evidence is correct but the answer is wrong, investigate prompt/context construction, model behavior and grounding.

Agentic RAG

The system can decide whether retrieval is needed, which source to query, reformulate a question, retrieve again, or stop.

Study checkpoint: Explain this topic in 60 seconds, draw the architecture from memory, then name two failure modes and two production mitigations.

5. Vectorless RAG / PageIndex-style Retrieval

Definition

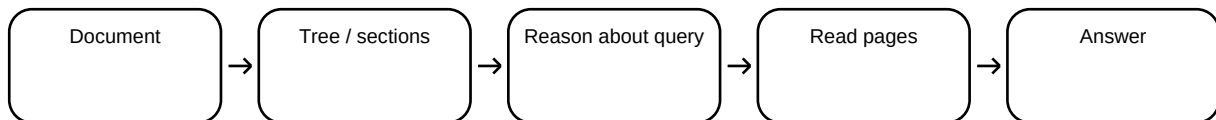
Vectorless RAG is a family of retrieval approaches that use document structure, hierarchy and reasoning rather than relying primarily on embedding similarity. A long document can be represented as a tree of sections/pages and navigated using the question.

Why it matters

Embeddings are excellent for semantic similarity, but structured documents such as annual reports, manuals and policy books often have meaningful headings, tables, page relationships and section hierarchy.

Architecture

Structure-aware / vectorless RAG



Comparison

Vector RAG: embedding → nearest chunks. Vectorless/structure-aware: document → hierarchical index/tree → reason about relevant sections → read selected pages/sections.

When to use

Use when document structure carries strong meaning, especially long reports, technical manuals, financial filings and hierarchical knowledge. Hybrid approaches can combine both.

Study checkpoint: Explain this topic in 60 seconds, draw the architecture from memory, then name two failure modes and two production mitigations.

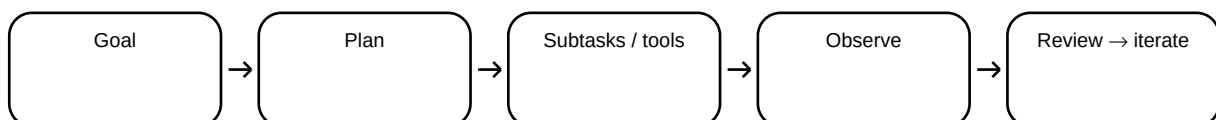
6. Deep Agents

Definition

A deep agent is designed for longer, multi-step work: planning, decomposition, context engineering, file/tool use, sub-agents, iterative review and long-running execution.

Architecture

Deep agent loop



Example

Research agent: clarify goal → plan subtasks → delegate research → collect evidence → synthesize → critique → revise → produce report.

When not to use

If a deterministic API call or short RAG pipeline solves the task, a deep agent can add unnecessary cost, latency and failure modes.

Production concerns

Step limits, budgets, timeouts, durable state, tool permissions, observability, human approval and evaluation.

Study checkpoint: Explain this topic in 60 seconds, draw the architecture from memory, then name two failure modes and two production mitigations.

7. Guardrails — Security and Safety

Definition

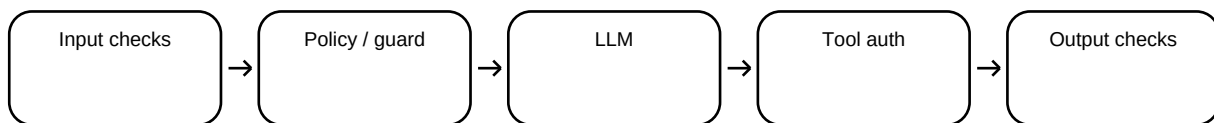
Guardrails are controls around model and tool execution that prevent unsafe, invalid, off-topic, policy-violating or high-risk behavior.

Threats

Prompt injection; jailbreaks; PII leakage; malicious tool arguments; data exfiltration; unsafe output; excessive tool use; unauthorized actions.

Architecture

Guardrail defense



Defense in depth

Input validation → policy/classification → model → tool authorization → output validation → logging/trace → human approval for high-risk actions.

Key rule

Never rely on the model alone to enforce authorization. The backend/tool layer must independently verify identity, permissions and allowed operations.

Study checkpoint: Explain this topic in 60 seconds, draw the architecture from memory, then name two failure modes and two production mitigations.

8. LLM Evaluation — Measure, Don't Guess

Definition

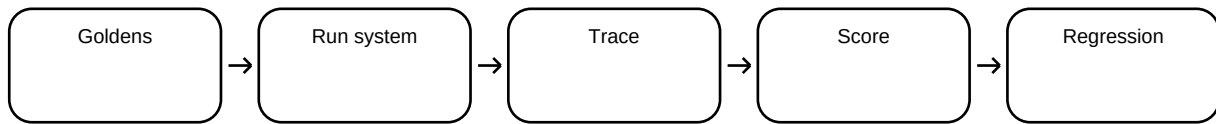
LLM evaluation is systematic measurement of model, retrieval, agent and end-to-end behavior against a fixed dataset or benchmark.

RAG metrics

Faithfulness/groundedness; answer relevancy; context precision; context recall; answer correctness. Also track latency, token usage, cost and safety.

Architecture

Evaluation loop



Evaluation loop

Create goldens → run system → collect traces → score retrieval → score answer → compare to baseline → investigate failures → improve → regression test.

LLM-as-a-judge

Useful for semantic qualities, but judge prompts and model bias matter. Combine judge scores with deterministic checks and human-reviewed samples.

Study checkpoint: Explain this topic in 60 seconds, draw the architecture from memory, then name two failure modes and two production mitigations.

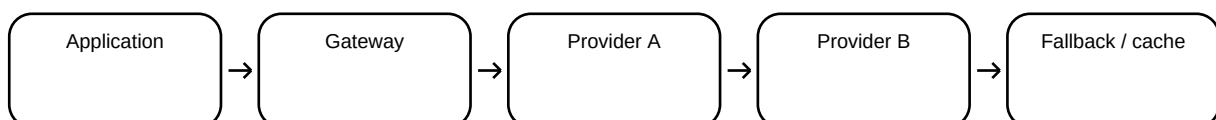
9. LLM Gateways — Production Model Infrastructure

Definition

An LLM gateway is middleware between an application and multiple model providers. The linked tutorial demonstrates a LiteLLM-based gateway with unified calls, routing/fallbacks, caching, rate limiting, observability and cost tracking.

Architecture

LLM Gateway



Capabilities

Provider abstraction; routing; fallbacks; retries; caching; rate limiting; cost tracking; logging; model switching.

Code pattern

```
from litellm import completion

providers = [
    ("openai", "gpt-4o-mini"),
    ("groq", "groq/llama-3.3-70b-versatile"),
]
for name, model in providers:
```

```
try:
    response = completion(
        model=model,
        messages=[{"role": "user", "content": "Explain RAG in one sentence."}]
    )
    print(name, response.choices[0].message.content)
except Exception:
    # In production: record the failure and route to a fallback.
    pass
```

Production rule

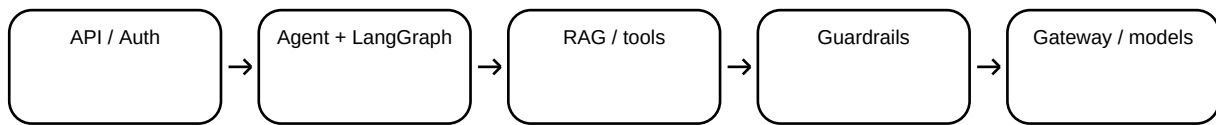
Keep provider credentials and routing policy outside application business logic. A gateway should make provider failure and model changes operationally manageable.

Study checkpoint: Explain this topic in 60 seconds, draw the architecture from memory, then name two failure modes and two production mitigations.

10. End-to-End Agentic RAG Reference Architecture

A strong production design combines the course topics instead of treating them as isolated libraries.

Reference architecture



Recommended flow: authenticate → validate input → route/classify → retrieve or call tools → reason/plan → execute approved actions → validate output → trace → evaluate. High-risk actions should pause for human approval.

Example: Enterprise support assistant

Layer	Responsibility
Frontend	Chat UI, streaming, citations, approval UI
API	Auth, rate limits, request validation
Agent	Planning, tool selection, state transitions
RAG	Private documents, metadata filtering, retrieval/reranking
Tools	CRM/order/DB operations with server-side authorization
Guardrails	Prompt injection, PII, policy, output checks
Gateway	Provider routing, fallback, caching, cost controls
Observability	Traces, tool calls, latency, errors, token/cost
Evals	Regression dataset and quality/safety metrics

Production checklist

■ Authentication and authorization are independent of the LLM. ■ Tool calls have schemas and timeouts. ■ Agent has maximum steps. ■ Retrieval has metadata/access filters. ■ Prompt injection is treated as untrusted data. ■ High-risk actions require approval. ■ Traces capture model/tool/retrieval events. ■ Fixed eval set catches regressions. ■ Provider fallback exists for critical workloads. ■ Cost and latency budgets are monitored.

11. Practical Mini-Projects

Project A — Customer Support RAG Agent

Build a PDF/document chatbot with metadata, citations, hybrid retrieval and a LangGraph agent that decides whether to retrieve, call an order-status tool, or ask for clarification.

Project B — Refund Approval Agent

Graph: classify request → fetch order → eligibility → risk score → human approval for high-value refunds → refund API → audit log. Add retry/error branches and checkpoints.

Project C — Research Deep Agent

Planner creates subtasks; researcher agents gather evidence; reviewer checks claims; final agent writes a report. Enforce step, token and tool budgets.

Project D — Multi-provider gateway

Expose one internal model interface. Route cheap classification tasks to a low-cost model, high-quality tasks to a stronger model, and use fallback providers on failure. Record cost and latency.

12. Fast Revision Sheet

Term	Remember it like this
LLM	Generates/reasons over language and structured output
Prompt	Instructions + context sent to the model
Tool	Callable function/API the model can request
Agent	Model + tools + decision loop
LangChain	Application building blocks
LangGraph	Explicit stateful workflow orchestration
RAG	Retrieve external knowledge before generation
Embedding	Maps text into vectors for semantic similarity
Vector DB	Stores/searches embeddings
Vectorless RAG	Uses document structure/reasoning instead of primarily vectors
Deep Agent	Long-running planning/delegation workflow
Guardrail	Safety/policy control around AI execution
HITL	Human approval/input in the workflow
Eval	Measure system quality against evidence
LLM Gateway	Central model routing/reliability/cost layer

Memory sentence: **RAG gives knowledge. Tools give abilities. LangGraph gives control. Guardrails give safety. Evals give confidence. Gateways give reliability.**

13. Sources & How This Guide Was Built

Primary course: Krish Naik, "Complete Agentic AI Course In 10 Hours- Langchain, Langgraph, RAG, Vectorless RAG, Guardrails, Evals." The published course page lists the nine major timestamped sections from Introduction through LLM Gateways. The indexed transcript is approximately 120,259 words.

The linked repositories were inspected for the course implementation structure, including the LangChain v1 notebooks, LangGraph course repository, RAG tutorials, PageIndex/vectorless RAG notebook, guardrails notebook, RAG evaluation notebook, and LLM gateway notebook. The gateway notebook explicitly demonstrates a LiteLLM + LangChain approach with routing/fallback/caching/cost-tracking themes.

Important: this is an original study guide and synthesis. It does not reproduce the video transcript or repository notebooks verbatim. Code examples are compact teaching patterns intended for study and should be adapted to the current library versions.

Source links:

YouTube: <https://www.youtube.com/watch?v=rV3HJ4LEZ7k>

LangChain repo: <https://github.com/krishnaik06/Langchain-V1-Crash-Course>

LangGraph repo: <https://github.com/krishnaik06/Agentic-LanggraphCrash-course>

RAG repo: <https://github.com/krishnaik06/RAG-Tutorials>

Vectorless RAG notebook: [https://github.com/krishnaik06/RAG-Tutorials/blob/main/PageIndex_Vectorless_RAG_CrashCourse%20\(1\).ipynb](https://github.com/krishnaik06/RAG-Tutorials/blob/main/PageIndex_Vectorless_RAG_CrashCourse%20(1).ipynb)

Guardrails notebook:

https://github.com/krishnaik06/Langchain-V1-Crash-Course/blob/main/updatedlangchain/langchain_guardrails_crash_course.ipynb

RAG evaluation notebook: https://github.com/krishnaik06/RAG-Tutorials/blob/main/1-rag_evaluation.ipynb

LLM gateway notebook: https://github.com/krishnaik06/Langchain-V1-Crash-Course/blob/main/llm_gateway_tutorial.ipynb